

SENA — BIENESTAR INSTITUCIONAL

Bienestar SENA

Sistema de Gestión de Citas

Manual Técnico

Documentación técnica completa del sistema: arquitectura, base de datos, seguridad RBAC, Edge Functions y guía de despliegue.

Versión 1.0 · 6 de julio de 2026

1. Descripción del Sistema

Bienestar SENA es una aplicación web SaaS para la gestión de citas de bienestar institucional del SENA. Permite a aprendices agendar citas con los profesionales de Psicología, Enfermería y Trabajo Social, y ofrece herramientas de coordinación, reportes y administración para el personal.

Características principales

- Agendamiento de citas en línea con validación de disponibilidad en tiempo real
- Control de acceso por roles (RBAC) con 7 roles y 26 permisos granulares
- Notificaciones por correo electrónico al confirmar y recordar citas
- Recordatorios automáticos 24 horas antes de cada cita
- Panel de reportes con estadísticas de atención por dependencia y período
- Control de fichas activas: el administrador sube el padrón del SENA y valida el registro de aprendices
- Encuesta de satisfacción al finalizar cada atención
- Exportación del historial de citas a PDF

2. Arquitectura General

El sistema sigue una arquitectura SPA + BaaS (Single Page Application + Backend as a Service). No requiere servidor Node.js propio en producción.

Capas del sistema

FRONTEND	React 19 + Vite 6 — SPA servida por Vercel CDN
BACKEND	Supabase: PostgreSQL 15 + Auth + Storage + Edge Functions (Deno)
DESPLIEGUE	Vercel (frontend) + Supabase cloud (backend)
CORREO	Resend API — envío de emails transaccionales
REPOSITORIO	GitHub — CI/CD automático con Vercel en cada push a main

Flujo de datos

- El navegador carga la SPA desde Vercel CDN
- La SPA se comunica directamente con Supabase usando la API REST y realtime
- Las RLS (Row Level Security) de PostgreSQL garantizan que cada usuario solo acceda a sus datos
- Las Edge Functions (Deno) corren en los servidores de Supabase para lógica server-side (emails)
- Vercel no ejecuta código server-side — es puramente CDN para archivos estáticos

3. Stack Tecnológico

Frontend

Tecnología	Versión	Uso
React	19.x	Framework UI — componentes y estado
Vite	8.x	Bundler y servidor de desarrollo
React Router DOM	7.x	Enrutamiento del lado del cliente (SPA)
@supabase/supabase-js	2.x	Cliente oficial de Supabase
Lucide React	1.x	Iconografía
Recharts	3.x	Gráficas y reportes
Sonner	2.x	Toast notifications
SheetJS (xlsx)	0.18.x	Parsing de Excel para el padrón de fichas
date-fns	4.x	Formateo y cálculo de fechas

Backend / Infraestructura

Tecnología	Rol
Supabase PostgreSQL 15	Base de datos relacional con RLS
Supabase Auth	Autenticación — JWT, email/password
Supabase Edge Functions	Lógica server-side en Deno (emails)
Supabase Storage	Almacenamiento de archivos adjuntos
Resend	Servicio de envío de emails transaccionales
Vercel	Hosting y CDN del frontend
GitHub	Control de versiones y CI/CD

Testing

Herramienta	Uso
Vitest 4.x	Test runner (147 tests — todos pasan)
@testing-library/react 16.x	Tests de componentes React
@testing-library/jest-dom 6.x	Matchers DOM extendidos
jsdom 29.x	Entorno DOM simulado en Node.js

4. Estructura de Carpetas

El proyecto sigue arquitectura Feature-Based — cada módulo tiene su propia carpeta con componentes, páginas y lógica relacionada.

Carpeta	Contenido
src/features/auth/	Login, registro, onboarding, perfil
src/features/appointments/	Dashboard, detalle, historial, atención
src/features/admin/	Panel admin, usuarios, roles, fichas
src/features/dashboard/	Dashboard de coordinación
src/features/reports/	Reportes y estadísticas
src/features/professional/	Agenda, notas y estadísticas profesional
src/shared/	Layout, componentes reutilizables
src/providers/	AuthProvider, AppointmentModalContext
src/routes/	AppRoutes, ProtectedRoute
src/lib/	Cliente Supabase, notificaciones, permisos
src/test/	Suites de pruebas automatizadas
supabase/functions/	Edge Functions: notify-appointment, send-reminders
scripts/	Generadores de documentos (manuales, testing)

5. Base de Datos

PostgreSQL 15 en Supabase. Todas las tablas tienen Row Level Security (RLS) habilitado.

Tablas principales

Tabla	Descripción
profiles	Datos de perfil de cada usuario (extiende auth.users)
roles	Catálogo de roles: APRENDIZ, PSICOLOGIA, ENFERMERIA, etc.
permissions	Catálogo de 26 permisos granulares del sistema
role_permissions	Relación N:M entre roles y permisos
dependencies	Dependencias de Bienestar: Psicología, Enfermería, Trabajo Social
appointments	Citas: estado, fecha, motivo, profesional, notas clínicas
system_settings	Configuración global: ubicación, whitelist_enabled, etc.
aprendiz_whitelist	Padrón de aprendices activos (cédula + ficha)
satisfaction_surveys	Encuestas de satisfacción post-atención (rating 1-5)
programs	44 programas de formación SENA activos

Ciclo de vida de una cita (campo status)

Estado	Descripción
pending	Cita creada por el aprendiz, esperando confirmación del profesional
confirmed	Profesional confirmó la cita — aparece en su agenda
in_progress	Atención iniciada — el cronómetro está corriendo
completed	Atención finalizada — el aprendiz puede dejar encuesta
cancelled	Cancelada por aprendiz o coordinación
no_show	El aprendiz no se presentó a la cita

Las citas solo las puede crear el APRENDIZ. El profesional confirma, inicia y completa. La coordinación puede cancelar cualquier cita activa.

6. Sistema RBAC (Control de Acceso por Roles)

El sistema implementa RBAC con 7 roles y 26 permisos. Los permisos se verifican tanto en el frontend (UI condicional) como en el backend (RLS de PostgreSQL).

Roles y acceso

Rol	Home	Permisos clave
APRENDIZ	/dashboard	Ver y cancelar sus propias citas, agendar citas
PSICOLOGIA	/professional	Confirmar, iniciar, completar citas, notas clínicas
ENFERMERIA	/professional	Idéntico a PSICOLOGIA
TRABAJO_SOCIAL	/professional	Idéntico a PSICOLOGIA
COORDINACION	/coordination	Ver todas las citas, cancelar cualquiera, exportar fichas
ADMINISTRADOR	/admin	CRUD usuarios, gestionar dependencias, fichas, taggle
SUPERADMIN	/admin	Todos los permisos — acceso total al sistema

Implementación técnica

- Los permisos se definen en `src/lib/permissions.js` como constantes (`PAPPOINTMENTS_CREATE`, etc.)
- El hook `useCan()` verifica si el rol actual tiene un permiso específico
- Las RLS de PostgreSQL implementan la misma lógica en el servidor como segunda capa de defensa
- Las rutas protegidas usan `ProtectedRoute` con `requiredRoles`

7. Edge Functions (Deno)

Las Edge Functions corren en los servidores de Supabase usando el runtime Deno. No requieren Node.js.

notify-appointment

- Trigger: el frontend la llama al confirmar una cita
- Función: envía email de confirmación al aprendiz vía Resend
- Resolución de email: usa la service-role key para leer auth.users.email (profiles no tiene email)
- Versión activa: v6

send-reminders

- Trigger: llamada externa (cron manual o programado)
- Función: envía recordatorios 24h antes a aprendices con cita al día siguiente
- Autenticación: acepta service-role Bearer o header x-cron-secret
- Prevención de duplicados: columna appointments.reminder_sent = true
- Versión activa: v1

*Para activar los emails en producción se debe configurar el secret RESEND_API_KEY en Supabase Dashboard !'
Edge Functions !' Secrets.*

8. Sistema de Control de Fichas (Whitelist)

El administrador puede cargar el padrón oficial del SENA con los aprendices activos. Cuando la validación está activada, solo los aprendices que aparezcan en el padrón pueden registrarse e iniciar sesión.

Tabla aprendiz_whitelist

Columna	Tipo	Descripción
id	UUID	Clave primaria
document_number	TEXT	Número de cédula del aprendiz
ficha_number	TEXT	Número de ficha del programa
full_name	TEXT	Nombre completo (opcional)
program	TEXT	Programa de formación (opcional)
uploaded_at	TIMESTAMPZ	Fecha de importación
uploaded_by	UUID	Usuario que importó el registro

Flujo de validación

- 1. ADMIN/COORDINACION sube CSV o Excel con columnas: cedula, ficha, nombre (opc), programa (opc)
- 2. El sistema importa en lotes de 500 con upsert (sin duplicados)
- 3. ADMIN activa el toggle whitelist_enabled en system_settings
- 4. Al registrarse: se valida cédula + ficha antes del signUp
- 5. Al hacer login: si el rol es APRENDIZ, se valida contra el padrón activo
- 6. Si no aparece !' se hace signOut inmediato y se muestra mensaje de error

RLS aplicadas

- whitelist_read_public: lectura pública (necesario para validar antes del signUp)
- whitelist_write_staff: escritura solo para COORDINACION, ADMINISTRADOR y SUPERADMIN
- public_read_whitelist_setting: solo la clave whitelist_enabled es legible sin sesión

9. Despliegue en Producción

Frontend — Vercel

- Repositorio: github.com/Arley0113/gestion-citas
- Rama de producción: main
- CI/CD: cada push a main dispara un deploy automático en Vercel
- URL de producción: <https://gestion-citas-nu.vercel.app>
- Framework preset: Vite (Vercel lo detecta automáticamente)

Variables de entorno en Vercel

Variable	Descripción
VITE_SUPABASE_URL	URL del proyecto Supabase
VITE_SUPABASE_ANON_KEY	Llave anónima pública de Supabase

Backend — Supabase

PROYECTO ID	hopjfppngueuhwuakzwf
REGIÓN	sa-east-1 (São Paulo)
PLAN	Free tier

Secrets de Edge Functions (Supabase Dashboard)

Secret	Uso
RESEND_API_KEY	Envío de emails — requerido para que los emails funcionen
CRON_SECRET	Autenticación del cron de recordatorios (opcional)
SUPABASE_SERVICE_ROLE_KEY	Auto-disponible en Edge Functions

Sin RESEND_API_KEY configurado, las Edge Functions corren sin error pero no envían emails. Configurarlos en Dashboard !' Edge Functions !' Secrets.

10. Testing Automatizado

El proyecto cuenta con 147 tests automatizados organizados en 4 suites. Todos pasan en el entorno de CI.

Archivo	Tests	Qué cubre
rbac/permissions.test.js	59	Permisos por rol, escalación de privilegios, exclusividad
utils/formatters.test.js	33	formatTime, timeLabel, duración de citas, estados
business/appointments.test.js	38	Filtros, mood, onboarding, encuesta de satisfacción
security/notifications.test.js	17	Sanitización XSS, autenticación de send-reminders

Comandos

- `npm test` — ejecuta todos los tests una vez
- `npm run test:watch` — modo watch, re-ejecuta al guardar
- `npm run test:coverage` — genera reporte de cobertura en `/coverage`

Los tests no requieren variables de entorno ni conexión a Supabase. Se ejecutan offline.